

DAA -01

(20/01/25)

Algorithm:

→ Seq of steps to solve a prob

→ Sorting:

(inc, dec, lexicograph)

1) Search

$$\begin{matrix} \text{Best (1)} \\ \swarrow \quad \searrow \\ \text{Success} < \text{Worst (n)} \end{matrix} \quad \text{Avg} \approx n-1 \approx n$$

↳ unsuccess (n)

→ Refine linear search

i) sort.

↳ If greater val found while
linear search $\Rightarrow$ search unsuccess.

ii) sort + binary search
 $\log(n)$

↳ sorting ($N, N^2, n \log n$).

Insertion sort (iterative)

WL $\rightarrow n^2$, BL $\rightarrow n$

Merge sort (recursive)

WL, BL $\rightarrow n \log n$ → for memory constraint $\Rightarrow$ insertion sort

DATE: _____

DATE: _____

PC1: 10s

PC2: 100s

↳ tell worst case?

Tester → PC3: 1000s.

⇒ Algorithm Analysis:

↳ machine independent.

↳ language independent.

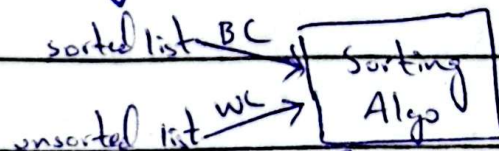
↳ based on algo/pseudo code.

↳ based on time & asymptotic behaviour.

↳ estimation

$(O, \pi, 0)$

2) Sorting:



(Avg) partial sorted list

Insertion Sort

BC → n

WC → n^2

Avg → n^2

Merge Sort

BC → $n \log n$

WC → $n \log n$

Avg → $n \log n$

Algo:

1) Seq of steps to solve a prob.

2) Algo must halt. (with correct O/P)

3) Unambiguous steps

4) Handle edge cases or clearly state unhandled cases.

$$n^2 + n - 1 \quad (\text{run-time})$$

$$n^2 \rightarrow \text{Big O}$$

$$\rightarrow \text{Omega } \Omega$$

$$\rightarrow \Theta$$

DAA-02

(24/01/25)

Analysis of Algo:

Algorithm (Program independent)

↳ Seq of steps

↳ unambiguous steps/statements

↳ Must HALT (with correct O/P)

↳ Edge case (handled)

↳ Mathematical tool $\rightarrow$ must be correct.

↳ used for analysis

↳ mathematical induction.

• space

↳ proofs.

• time

$\rightarrow$ Heap (Priority Queue) (Heap sort Algo). $n \log n$.

Limitation:

$\rightarrow$ can't go beyond specified.

DATE _____

DATE _____

↳ space complexity $\Rightarrow$ extra memory used

↳ time complexity $\Rightarrow$ runtime

$\Delta t \rightarrow$ change in time

Problem size $\propto$ Time with respect to

Design: increase in size

↳ Divide & Conquer

↳ Dynamic

↳ Greedy

$\Rightarrow$ Greatest Common Divisor Problem:

1) Euclidean:

$GCD(m, n)$

if $(m \% n) == 0$ return n

else return $GCD(n, m \% n)$

2) 60, 12

↳ if small value divides both $\Rightarrow GCD$

else decrement small value & check again.

$60 \% 12 == 0$ & $12 \% 12 == 0$

↳ $GCD(60, 12) \Rightarrow 12$

prob: If m & n are relatively prime & extremely large.

$\Rightarrow$ if relatively prime $\Rightarrow$ return 1

↳ Euclidean is better.

3) $\hookrightarrow$ common prime factors.

$\hookrightarrow$ requires list of prime factors.

$\hookrightarrow$ sieve of Eratosthenes (Algo for P.M b/w 2 nums)

$$\begin{array}{l} 24 \rightarrow 2 \cdot 2 \cdot 2 \cdot 3 \\ 18 \rightarrow 2 \cdot 3 \cdot 3 \end{array}$$

$$n = 25$$

(Skip 1) $\underline{1}$ 2 3 4 5 6 7 8 9
 $\cancel{10}$ 11 $\cancel{12}$ 13 $\cancel{14}$ 15 $\cancel{16}$
 17 $\cancel{18}$ 19 $\cancel{20}$ $\cancel{21}$ $\cancel{22}$ 23
 $\cancel{24}$ $\cancel{25}$

1) Eliminate factors

$\hookrightarrow p \Rightarrow$ first eliminate $p \cdot p$

$2 \rightarrow 4$ | $3 \rightarrow 9$

$\sqrt{n}$ iterations req.

$$\sqrt{25} = 5$$

$\Rightarrow$ If not perfect square $\Rightarrow \lfloor \sqrt{n} \rfloor$

$$N/2, N/3, N/5$$

$$\left(\frac{n}{2}, \frac{n}{3}, \frac{n}{5}, \dots \right)$$

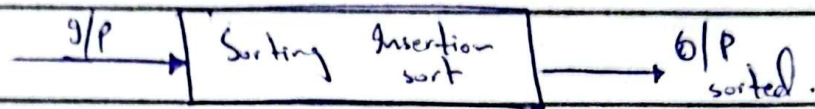
$$n \left(\frac{1}{2}, \frac{1}{3}, \frac{1}{5}, \dots \right)$$

$\boxed{\quad}^{n \times n}$
 (initialized with 1)

Harmonic Progression.

$$\sqrt{n} (\log \log n)$$

↳ Euclidean is better in a sense
it does handle relatively prime (1 iteration)



Input Types

- Best Case — sorted
- Worst Case — Rev sorted
- Avg Case — Semi sorted.

int $i = 0$ → unit time

$a = b + c$ → unit time

for $i = 0 - m$ } n -unit time
 $c = 5$

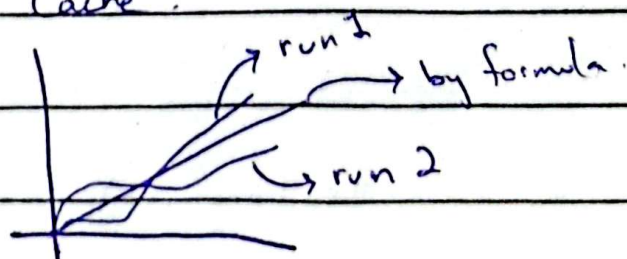
ALU → add, sub only (arithmetic)
comparison (logical unit)

int → 4 byte → 4 machine cycle.
or 2 machine cycles.

delay → distance & cycles.

i) Registers → fastest.

ii) Cache.



DATE: _____

DATE: _____

⇒ We cannot evaluate system

qualitative time.

$n^2 + n + 1$ → machine dependent.

↓
Asymptotic behaviour

→ Order of Growth.

→ Efficiency class.

- linear

→ quadratic

- cubic

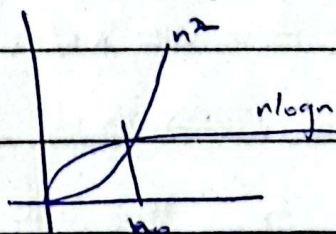
- polynomial

- exponential

- log.

n	n^2	kn^2
10^{10}	10^{20}	$10^{200} = 10^{180+20}$
10^{20}	10^{40}	$10^{200} = 10^{180+20}$
10^{40}	10^{80}	$10^{100} = 10^{20+80}$

Merge sort ⇒ $a \log n + bn + c$



⇒ merge sort beats insertion sort
after n_0 .

⇒ constants (c) is coming from
the state of machine at
the time.

Basic Operation DAA-03 (27/01/25)

same time magnitude of value matters

10 101024 (even or odd?)
 $\rightarrow$ takes more time

In sorting $\Rightarrow$ comparison operation

$$\Rightarrow T(n) = COP \cdot (n)$$

Total time Cost of Basic Operation (No. of times basic operation being executed)

$$\Rightarrow T(n) = C(n) \quad (\text{If } COP = 1)$$

Machine 'A' Algo 'X'

1) $A = 10 \text{ ins/s.}$

size-n $\rightarrow 1s$

$B = 100 \text{ ins/s.}$

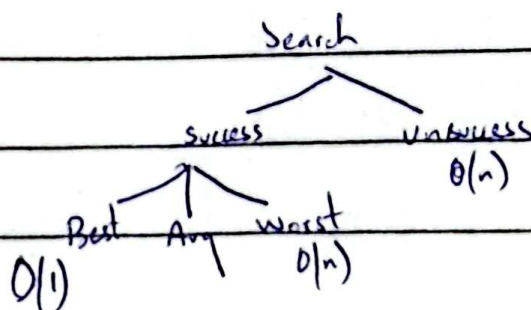
size-n $\rightarrow ?$ (0.1s)

2) $A = 100 \text{ ins/s.}$

size-n $\rightarrow 1s$

$B = 100,000 \text{ ins/s.}$

size-n $\Rightarrow ?$ (0.001 sec)



DATE: _____

DATE: _____

Sorting:

Insertion Sort		sorted $\rightarrow n$
		unsorted $\rightarrow n^2$
		semi-sorted $\rightarrow n^2$

Merge sort $O(n \log n)$

single element array (supposed BL)

Avg case has multiple possibilities.

$\Rightarrow$ Avg Analysis $\times$

$\Rightarrow$ Probability that element exists is p

~~~~~ does not exist  $1-p$

(Max no of comparisons)

successful + unsuccessful

$$1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + i \cdot \frac{p}{n} + \dots + n \cdot \frac{p}{n} + (1-p) \cdot n$$

$$\frac{p}{n} (1 + 2 + 3 + \dots + i + \dots + n) + (1-p) \cdot n$$

$$\frac{p}{n} \cdot \left( \frac{n(n+1)}{2} \right) + (1-p) \cdot n$$

$$\frac{p(n+1)}{2} + (1-p) \cdot n$$

$\Rightarrow P=0$   $0 + n$

$\Rightarrow P=1$   $\frac{n+1}{2} + 0$

DAA-04

|31/01/25)

1.2-2

$$8n^2 \leq 64n \lg n.$$

$$n \leq 8 \lg n.$$

$n=1$

$$1 \leq 8 \lg 1 \rightarrow < 8(0) \times$$

$n=2$

$$2 \leq 8 \lg 2 \rightarrow \leq 8 \checkmark$$

powers of  
2 ~~not 3~~

$$4 \leq 8 \lg 4 \rightarrow \leq 16 \checkmark$$

$$\lg_2 46 = \frac{\lg 46}{\lg 2}$$

$$8 \leq 8 \lg 8 \rightarrow \leq 24. \checkmark$$

$$16 \leq 8 \lg 16 \rightarrow \leq 32. \checkmark$$

$$32 \leq 8 \lg 32 \rightarrow \leq 40. \checkmark$$

$$64 \leq 8 \lg 64 \rightarrow \leq 48 \times$$

$$32 - 64 \rightarrow \text{mid} = 48 \text{ (random try)}$$

$$46 \leq 8 \lg 46.$$

$$46 \leq 44 \times$$

$$42 \leq 8 \lg 42 \rightarrow \leq 43 \checkmark$$

$$43 \leq 8 \lg 43 \rightarrow \leq 43.41 \checkmark$$

1-1 Comparison of running times:

$$f(n) \text{ ms} \rightarrow \lg n \text{ ms} = 1 \text{ s.}$$

$$\lg n 10^{-6} \text{ s} = 1 \text{ s}$$

$$\lg_2 n = 10^6$$

$$n = 2^{10^6}$$

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

## Insertion Sort:

- Decrease & conquer.

- Space complexity    $O(1)$

- Time Complexity  $\rightarrow O(n^2)$  worst case  
 $\rightarrow O(n)$  best case.

Avg <sup>case</sup> is taken as close to worst case.

$\rightarrow$  Place an element from unsorted part to sorted part of the array.

Bubble Sort: for  $1-n$ :  $\rightarrow n+1$  iterations  
 (Terminating check)  
 $\rightarrow$  loop body  $\rightarrow n$  times.

$\Rightarrow$  Cost is cost of the machine.

$\Rightarrow C_3=0 \rightarrow$  ~~comment~~ will be read but not passed <sup>as</sup> ~~to~~ the machine ins.

(Best Case:  
Time

$$T(n) = C_1 n + C_2(n-1) + 0 + C_4(n-1) + C_5(n-1) + C_8(n-1)$$

//  $C_6$  &  $C_7$  not executed.

$$= C_1 n + C_2 n - C_2 + C_4 n - C_4 + C_5 n - C_5 + C_8 n - C_8$$

$$= (C_1 + C_2 + C_4 + C_5 + C_8)n - (C_2 - C_4 - C_5 - C_8)$$

$$= a'n - b \Rightarrow \approx O(n)$$

AGE \_\_\_\_\_

DATE \_\_\_\_\_

Worst Case:

loop invariant

Mathematical Induction.

Base Step — True

Inductive Step.

IH — Assume True( $k$ )prove  $k+1$  $\Rightarrow$  one counter example  $\Rightarrow$  denied whole.Algo Correctness: $\rightarrow$  Initialization. $\rightarrow$  Maintenance. $\rightarrow$  Termination. loop invariant (finite)

Mathematical Induction (infinite).

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

Arg Case:

10 | 30 | 20 | 70

$t_i \Rightarrow$  avg time of while loop  
till  $i$

$$\sum_{i=2}^n t_i \equiv i$$

$$\sum_{i=2}^n t_i = 2 + 3 + \dots + n$$

$$= \frac{n(n+1)}{2} - 1$$

$$\sum_{i=2}^n = (t_i - 1)$$

$$\Rightarrow \sum_{i=1}^{n-1} i = \frac{n(n-1+1)}{2}$$

DAA -05

(12/02/2025)

Merge Sort.

(Inplace sorting).

↳ Divide & Conquer

↳ Recursive.

↳ Avg + Best + Worst  $\Rightarrow n \log n$

↳ loop invariant

↳ correctness

• proofing technique.

• mathematical induction.

condition/  
statement  
in algo

← Loop Invariant

1) Initialization.

2) Maintenance

3) Termination:

Insertion Sort:

↳ Decrease and Conquer.

↳ Iterative / Increment.

↳ Best — linear ( $n$ )

↳ Worst/Avg — quadratic ( $n^2$ )

loop invariant

(1-i-1)

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

## Merge Sort:

// Algo

Analysis:

Divide  $\rightarrow$  constant

Conquer  $\rightarrow$  recursive.  $2T(n/2)$

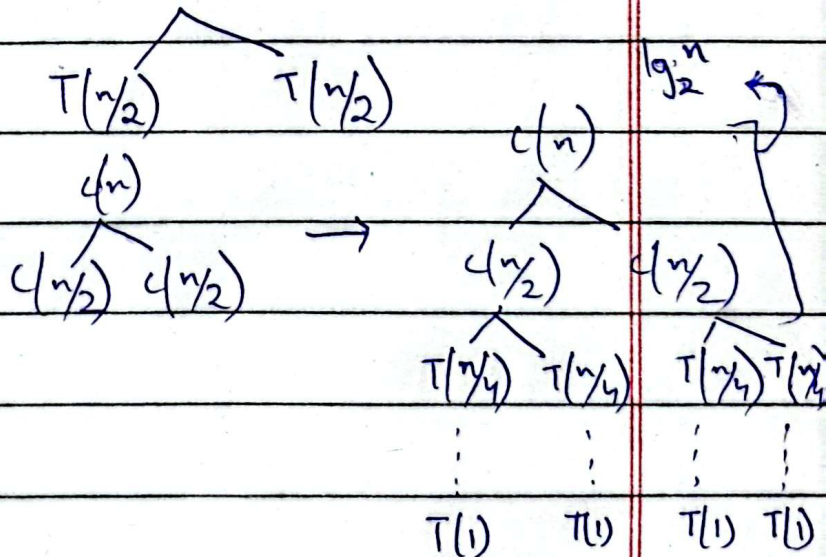
combine  $\rightarrow$   $n$ .

$$aT(n/b)$$

$a \rightarrow$  no of sub problems

$b \rightarrow$  size of sub problems.

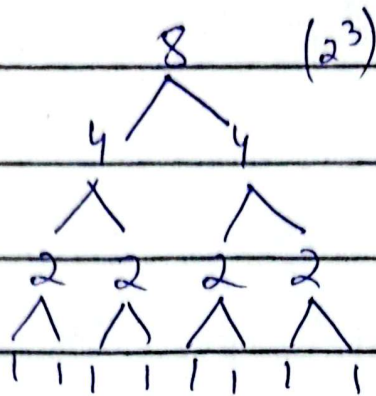
$$T(n) \rightarrow c(n)$$



$$\Rightarrow n * \lg_2 n = n \lg_2 n$$

DATE \_\_\_\_\_

DATE \_\_\_\_\_



$$\lg_2 n \Rightarrow \lg_2 2^3$$

$$\Rightarrow 3$$

$$\Rightarrow \text{level} = 3$$

$$T(n) = \begin{cases} c_0 \\ c_1 + 2T(n/2) + c_3 n \\ c_1 + c_2 n \lg n + c_3 n \end{cases}$$

$$O(n \lg n)$$

DAA - 06

(14/02/25)

## Asymptotic Notation:

$\left\{ \begin{matrix} O \\ \Omega \\ \sim \end{matrix} \right\}$   
tight bound

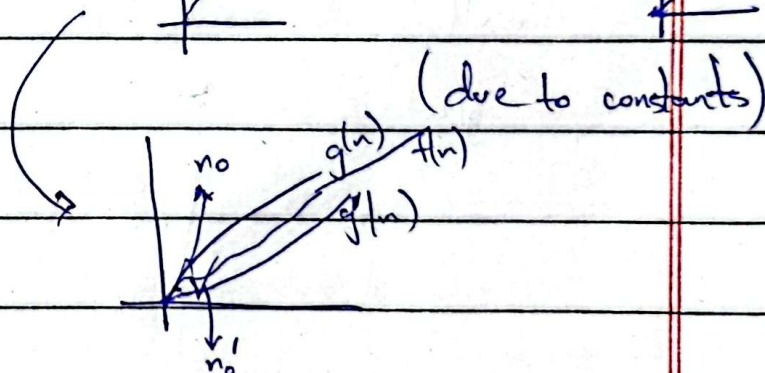
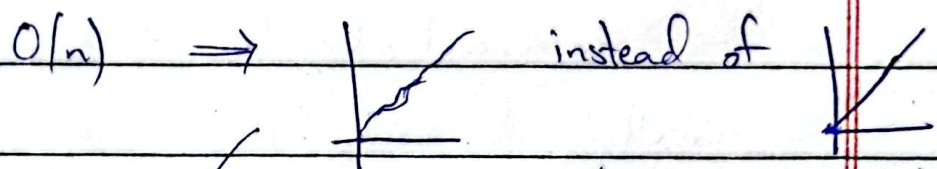
↳ due to different languages/systems.

for  $i=1$  —  $(n^2 + 2n - 6)$

for  $j=1$  —  $(n+8)$

↳  $\Theta(n^2 + 2n - 6)(n+8)$

↳  $O(n^3)$



$g(n) \rightarrow$  upper bound  
 $g'(n) \rightarrow$  lower bound.

after  $n_0$   $g(n) > f(n)$

after  $n'_0$   $g'(n) < f(n)$

Upper bound

$$f(n) \leq c g(n)$$

$\Rightarrow O$  (worst case)



time increasing

Lower bound

$$f(n) \geq c g(n)$$

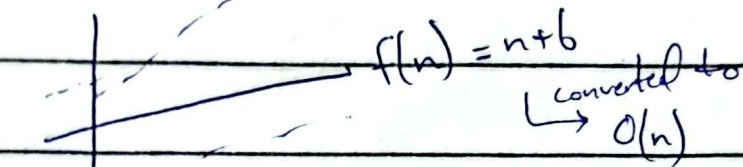
$\Rightarrow \Omega$  (best case)

DATE \_\_\_\_\_

DATE \_\_\_\_\_

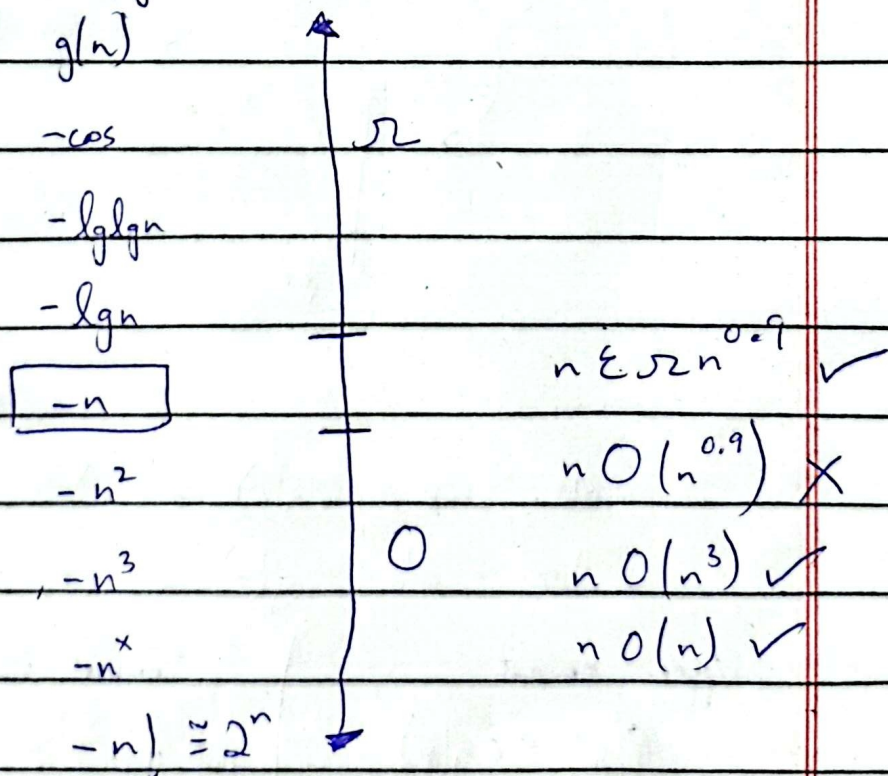
$$\left. \begin{array}{l} f(n) O(g(n)) \\ \rightarrow f(n) \leq c g(n) \\ n_0 \leq n \end{array} \right| \left. \begin{array}{l} f(n) \Omega(g(n)) \\ \rightarrow f(n) \geq c g(n) \\ n_0 \leq n \end{array} \right.$$

Why constant?



to convert this line to upper bound

multiply it with  $c$  i.e. 10.



$\Rightarrow \Theta \Rightarrow$  If upper & lower bound are same.

$\Rightarrow$  We have to find  $c$  &  $n_0$  values.

Practice:

1)  $n^2 + 4n + 6 \in O(n^2)$

Upper bound:  $n^2 + 4n + 6 \leq 2n^2$  valid for  $[2.5n^2]$

Lower bound:  $n^2 + 4n + 6 \geq \frac{1}{2}n^2$  valid for  $[n^2]$

$O(n^2) \quad n^2 \geq n^2 + 4n + 6$

$\Omega(n^2) \quad n^2 \leq n^2 + 4n + 6$

$\Rightarrow$  constants can be fractional but  
not negative.

finding  $c$  for  $O \Rightarrow n^2 + 2n + 6 \leq 2n^2$

$(n=1) \quad 11 \leq 2 \quad \times$

$(n=2) \quad 4+4+6 \leq 8 \quad \times$

$(n=5) \quad 25+20+6 \leq 50 \quad \times$

$\Rightarrow (n=10) \quad 100+40+6 \leq 200 \quad \checkmark$

2)  $2n^2 + 3n + 4 \in O(n^2)$

use L.H.S or R.H.S.

$$2n^2 + 3n + 4 \leq 2n^2 + 3n^2 + 4n^2$$

$$\leq 9n^2$$

valid for  $\hookrightarrow$ 

$3n \leq 3n^2$

$4 \leq 4n^2$

$1 \leq n$

$1 \leq n^2$

$1 \leq n$

$1 \leq n$

DATE \_\_\_\_\_

DATE: \_\_\_\_\_

⇒ (Attach high order term from big O with lower order terms)

⇒ (If  $f(n^2) \in O(n) \Rightarrow \text{stop } \times$ )

→ Issue comes  $\Rightarrow c$  is much greater &  $n$  is much smaller.

$$3) \quad 2n^2 + 3n + 4 \in O(n^4)$$

$$2n^2 + 3n + 4 \leq 2n^4 + 3n^4 + 4n^4 \\ \leq 9n^4.$$

|                  |                |                |
|------------------|----------------|----------------|
| $2n^2 \leq 2n^4$ | $3n \leq 3n^4$ | $4n \leq 4n^4$ |
| $1 \leq n^2$     | $1 \leq n^3$   | $1 \leq n^4$   |
| $1 \leq n$       | $1 \leq n$     | $1 \leq n$     |

↪  $n \geq 1$

$$4) \quad \log_4(2n^2 + 3n + 4) \in O(\log_4 n)$$

$$\log_4(2n^2 + 3n + 4) \leq \log_4(2n^2 + 3n^2 + 4n^2) \\ \leq \log_4(9n^2)$$

|               |                |                              |
|---------------|----------------|------------------------------|
| $4n^2 \geq 4$ | $3n \leq 3n^2$ | $\leq \log_4 9 + \log_4 n^2$ |
| $n \geq 1$    | $1 \leq n$     | ↪                            |

→ As log & exp are opposite

|                                  |                           |
|----------------------------------|---------------------------|
| $\log_4 9 \leq \log_4 n$         | ↪ $\log_4 n + 2 \log_4 n$ |
| $4^{\log_4 9} \leq 4^{\log_4 n}$ |                           |
| $\Rightarrow 9 \leq n$           |                           |

$$4) \log_9(6n^2 + 9n + 4) \in O(\log_9 n)$$

5)

$$6) 2n^2 + 3n + 4 \in \Omega(n^2)$$

 $\Omega$ 

$$2n^2 + 3n + 4 \geq n^2$$

↳ drop  
adding  
values  
(+ve only)

$$\geq 2n^2 \quad \checkmark$$

$$\geq n^2 \quad \checkmark$$

$$7) 2n^2 + 3n + 4 \in \Omega(n)$$

$$\geq 3n$$

$$8) 2n^2 - 3n + 4 \in \Omega(n^2)$$

$$\geq 2n^2 - 3n$$

$$\geq 2n^2 - 3n^2$$

$$(-ve \times) \geq -n^2$$

$$\geq 2n^2 - n^2$$

$$\Rightarrow \geq n^2$$

use the  
co-efficient here  
that does not  
make -ve

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

9)  $\sqrt{6n^3 + 7n^2 + 2n + 6}$   $O(n^{1.5})$

$$< \sqrt{6n^3 + 7n^3 + 2n^3 + 6n^3}$$

$$\leq \sqrt{21n^3}$$

$$\leq \sqrt{21} n^{1.5} \Rightarrow n^{1.5}$$

~~$O(n^{1.5})$~~

~~$O(n^{1.5})$~~

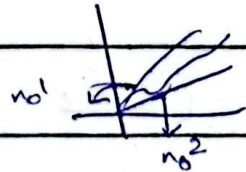
10) same  $O(n^{1.5})$

$$\geq \sqrt{6n^3}$$

$$\geq \sqrt{6} n^{1.5} \Rightarrow n^{1.5}$$

$\Rightarrow$  we use greater value of  $n^{1.5}$ .

as:



DAA - 07

(21/02/25)

Max-Value(A, n)

MaxValue = A[1].

for  $i = 1 \rightarrow n-1$

if  $A[i] > \text{MaxValue}$

MaxValue = A[i]

} 1 Basic Operation  
in loop

return MaxValue.

(executed every  
time)

$\Rightarrow$  Same best & worst case.

## Analysis Framework

→ Iterative Code

→ Recursive Code

Using Basic Operation:

$$T(n) = \text{Op} \cdot C(n)$$

$$\left\{ \begin{array}{l} \sum_{i=l}^u c a_i = c \sum_{i=l}^u a_i \\ \sum_{i=l}^u 1 = u - l + 1 \end{array} \right.$$

loop  $\Rightarrow$  summation  $\sum_{i=1}^{n-1} 1 = n - 1 + 1$

$$= n - 1 \Rightarrow O(n) \rightarrow O(n)$$

$\hookrightarrow \Omega(n) \rightarrow O(n)$

$\Rightarrow$  Unique Values(A, n)

for  $i = 0 \rightarrow n-2$

for  $j = 1 \rightarrow n-1$

if  $(A[i] == A[j])$  return false;

return true;

1) Successful

$$\hookrightarrow n^2$$

2) Unsuccessful

(Best)  $\hookrightarrow$  1st 2-ele duplicate.

(Worst)  $\hookrightarrow$  last 2 ele duplicate.

Best  $\rightarrow O(1)$

Worst  $\rightarrow O(n^2)$

$\Rightarrow$  Iterators are not basic Operations

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

↳ Basic Op Cost of Unique Values:

$$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i)$$

$$= \sum_{i=0}^{n-2} (n-1-i)$$

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

$$= (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-2+1)}{2}$$

$$= (n-1)(n-1) - \frac{(n-2)(n-1)}{2}$$

$$= \frac{2(n^2 - n - n + 1) - n^2 + n - 2}{2}$$

$$= \frac{2n^2 - 4n + 2 - n^2 + n - 2}{2}$$

$$= \frac{n^2 - n}{2} = \frac{1}{2} (n^2 - n)$$

$$O(n^2) \rightarrow \Theta(n^2)$$

$0+1+ \dots + n$   
 $\hookrightarrow \frac{n(n+1)}{2}$   
 $\hookrightarrow$  put upper bound  
 $n = n-2$   
 $\rightarrow \frac{(n-2)(n-2+1)}{2}$   
 $\frac{(n-2)(n-1)}{2}$

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

MatrixMul(A, B)

for  $i = 0 \rightarrow n-1$

for  $j = 0 \rightarrow n-1$

$C[i, j] = [0, 0]$

for  $k = 0 \rightarrow n-1$

$$C[i, j] = C[i, j] + A[i, k] * B[k, j]$$

BO-1                      BO-2                      BO-3

return C.

↳ Worst case can be simply found.

↳ Best case requires analysis.

↳ Not possible case (row  $\neq$  col)

↳ Iterative -  $n^3$

↳ Divide & Conquer  $\Rightarrow$  Concurrent processes -  
but in analysis, we  
consider/take single  
process only.

↳ Basic Operation

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (n-1-0+1)$$

$$= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n$$

DATE \_\_\_\_\_

DATE \_\_\_\_\_

$$= \sum_{i=0}^{n-1} n \sum_{j=0}^{n-1} 1 = \sum_{i=0}^{n-1} n/n$$

$$= n^3$$

Recursive Code:

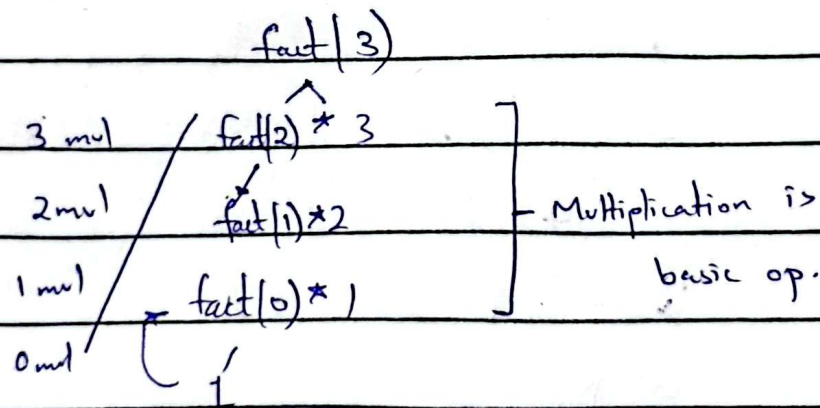
fact(n):

if (n==0) return 1;

(executed everytime but it is iterator)

else return f(n-1)\*n;

↳ problem is decreased by const value of 1



↳ convert recursive problem into recursive relation.

$$\text{fact}(n) = \text{fact}(n-1) + 1$$

$$\text{fact}(0) = 0 \quad (\text{initial Cond})$$

↳ Backward Substitution.

$$M(n) = M(n-1) + 1$$

$$= [M(n-2) + 1] + 1$$

$$= M(n-3) + 1 + 1 + 1$$

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

$$= M(n-4) + 1 + 1 + 1 + 1$$

$$\Rightarrow = M(n-i) + i$$

As we want  $M(0)$   $\rightarrow$  Base Cond

$$n-i = 0 \Rightarrow i = n$$

$$= M(n-n) + n$$

$$= M(0) + n$$

$$= 0 + n = n$$

### Tower of Hanoi Puzzle:

def transfer (discs, source, destination, storage):

if discs == 0 return

transfer

transfer.

$$\text{min\_discs} = 1$$

$$M(1) = 1 \quad (\text{initial cond})$$

$$M(n) = M(n-1) + 1 + M(n-1)$$

$\rightarrow$  not exactly  $n-1$ ,

but  $n-2$  & so on will cause unending recurrence rel

$$M(n) = 2M(n-1) + 1$$

$$M(n-1) = 2M(n-2) + 1$$

$$2^1 = 2M(n-1) + 1$$

$$= 2[M(n-2) + 1] + 1$$

$$2^2 = 4M(n-2) + 2 + 1$$

$$= 4[2M(n-3) + 1] + 2 + 1$$

DATE: \_\_\_\_\_

DATE: \_\_\_\_\_

$$2^3 = 8M(n-3) + 4 + 2 + 1$$

$$= 8[2M(n-4) + 1] + 4 + 2 + 1.$$

$$2^4 = 16M(n-4) + 8 + 4 + 2 + 1$$

$$\Rightarrow 2^i M(n-i) + 2^i - 1$$

$$= 2^{n-1} M(n-n+1) + 2^{n-1} - 1.$$

$$= 2^{n-1} M(1) + 2^{n-1} - 1$$

$$= 2^{n-1} + 2^{n-1} - 1$$

$$= 2 \cdot 2^{n-1} - 1$$

$$= 2^{n-1+1} - 1 = 2^n - 1$$

$$\hookrightarrow 0(2^n)$$